

Counter System Architecture

python

```
class ElementalSystem:
```

```
    """
```

```
    Implements Pancha Mahabhuta (Five Elements) philosophy  
    with additional mythological elements
```

```
    """
```

```
    ELEMENTS = {
```

```
        'FIRE': {'beats': ['WIND', 'ICE'], 'weak_to': ['WATER', 'EARTH']},  
        'WATER': {'beats': ['FIRE'], 'weak_to': ['WIND', 'THUNDER']},  
        'WIND': {'beats': ['WATER', 'EARTH'], 'weak_to': ['FIRE', 'EARTH']},  
        'EARTH': {'beats': ['THUNDER', 'WIND'], 'weak_to': ['WATER']},  
        'THUNDER': {'beats': ['WATER', 'METAL'], 'weak_to': ['EARTH']},  
        'SERPENT': {'beats': ['WIND', 'HUMAN'], 'weak_to': ['GARUDA']},  
        'GARUDA': {'beats': ['SERPENT'], 'weak_to': ['DIVINE']},  
        'DIVINE': {'beats': ['ALL'], 'weak_to': ['SUPREME_DIVINE']}
```

```
    }
```

```
    def resolve_combat(self, attacker, defender):
```

```
        """
```

```
        Resolve weapon combat based on elemental relationships  
        Returns: (attacker_damage, defender_damage, status_effects)
```

```
        """
```

```
        # Check if defender counters attacker
```

```
        if attacker.element in self.ELEMENTS[defender.element]['beats']:
```

```
            # Defender counters successfully
```

```
            return (0, defender.power, defender.status_effects)
```

```
        # Check if same element (mutual destruction for supreme weapons)
```

```
        elif attacker.element == defender.element:
```

```
            if attacker.tier == 'SUPREME' and defender.tier == 'SUPREME':
```

```
                return (0, 0, ['both_destroyed'])
```

```
            else:
```

```
                # Both deal damage
```

```
                return (attacker.power, defender.power, [])
```

```
        # Attacker wins
```

```
        else:
```

```
            return (attacker.power, 0, attacker.status_effects)
```


CHAPTER 7: ALGORITHM DESIGN & AI

Core Game Algorithms

1. Optimal Mantra Spending Algorithm

python

```
def calculate_optimal_mantra_usage(game_state, hand, available_mantra):
```

```
    """
```

Dynamic Programming approach to maximize value per mantra spent

Problem: Given N cards in hand with costs C1, C2, ..., CN and
values V1, V2, ..., VN, maximize total value with M mantra

This is a variant of the 0/1 Knapsack problem

```
    """
```

```
n = len(hand)
```

```
dp = [[0 for _ in range(available_mantra + 1)] for _ in range(n + 1)]
```

```
# Build table dp[][] in bottom-up manner
```

```
for i in range(1, n + 1):
```

```
    card = hand[i-1]
```

```
    cost = card.mantra_cost
```

```
    value = card.calculate_value(game_state) # Heuristic function
```

```
    for m in range(available_mantra + 1):
```

```
        if cost <= m:
```

```
            # Choose max of: include card vs exclude card
```

```
            dp[i][m] = max(
```

```
                value + dp[i-1][m-cost], # Include
```

```
                dp[i-1][m] # Exclude
```

```
            )
```

```
        else:
```

```
            dp[i][m] = dp[i-1][m]
```

```
# Backtrack to find which cards to play
```

```
cards_to_play = []
```

```
m = available_mantra
```

```
for i in range(n, 0, -1):
```

```
    if dp[i][m] != dp[i-1][m]:
```

```
        cards_to_play.append(hand[i-1])
```

```
        m -= hand[i-1].mantra_cost
```

```
return cards_to_play
```

```
def card_value_heuristic(card, game_state):
```

```
    """
```

Calculate card value based on game state

```
    """
```

```
base_value = card.power
```

```
# Situational bonuses
```

```
if game_state.opponent_life <= card.power:  
    base_value *= 3 # Lethal is highly valuable
```

```
if card.element in game_state.opponent_likely_elements:  
    base_value *= 0.5 # Likely to be countered
```

```
if card.has_status_effect:  
    base_value += 2 # Status effects add value
```

```
if game_state.turn_number <= 3 and card.cost >= 5:  
    base_value *= 0.3 # Expensive cards bad early
```

```
return base_value
```

Time Complexity: $O(N * M)$ where N = hand size, M = mantra available **Space Complexity:** $O(N * M)$

2. Counter Prediction Algorithm

```
python
```



```
class CounterPredictor:
```

```
    """
```

```
Machine learning approach to predict opponent's counter cards
```

```
Uses Bayesian inference updated with observed plays
```

```
    """
```

```
def __init__(self):
```

```
    self.opponent_deck_estimate = self.initialize_deck_probabilities()
```

```
    self.cards_seen = []
```

```
    self.elements_played = {'FIRE': 0, 'WATER': 0, 'WIND': 0, 'EARTH': 0}
```

```
def update_probabilities(self, card_played):
```

```
    """
```

```
Bayesian update based on observed card
```

```

$$P(\text{deck\_type} \mid \text{card\_played}) \propto P(\text{card\_played} \mid \text{deck\_type}) * P(\text{deck\_type})$$

```

```
    """
```

```
    self.cards_seen.append(card_played)
```

```
    self.elements_played[card_played.element] += 1
```

```
# Update deck type probabilities
```

```
for deck_type in self.opponent_deck_estimate:
```

```
    likelihood = self.get_likelihood(card_played, deck_type)
```

```
    prior = self.opponent_deck_estimate[deck_type]
```

```
    self.opponent_deck_estimate[deck_type] = likelihood * prior
```

```
# Normalize
```

```
total = sum(self.opponent_deck_estimate.values())
```

```
for deck_type in self.opponent_deck_estimate:
```

```
    self.opponent_deck_estimate[deck_type] /= total
```

```
def predict_counter(self, my_card):
```

```
    """
```

```
Predict probability opponent has counter to my card
```

```
    """
```

```
counter_element = self.get_counter_element(my_card.element)
```

```
# Calculate expected number of counter cards in opponent's hand
```

```
deck_type_probabilities = self.opponent_deck_estimate
```

```
total_counter_probability = 0
```

```
for deck_type, prob in deck_type_probabilities.items():
```

```
    # How many counter cards does this deck type typically run?
```

```
    typical_counters = self.get_typical_counters(deck_type, counter_element)
```

```

cards_left_in_deck = 40 - len(self.cards_seen)
hand_size = 5 # Assume average hand size

# Hypergeometric distribution
prob_counter_in_hand = self.hypergeometric_prob(
    population=cards_left_in_deck,
    successes_in_pop=typical_counters,
    sample=hand_size,
    successes_in_sample=1
)

total_counter_probability += prob * prob_counter_in_hand

```

```

return total_counter_probability

```

```

def hypergeometric_prob(self, population, successes_in_pop, sample, successes_in_sample):
    """
     $P(X = k) = \frac{C(K, k) * C(N - K, n - k)}{C(N, n)}$ 
    Where: N=population, K=successes_in_pop, n=sample, k=successes_in_sample
    """
    from math import comb
    if successes_in_sample > min(successes_in_pop, sample):
        return 0

    numerator = comb(successes_in_pop, successes_in_sample) * \
        comb(population - successes_in_pop, sample - successes_in_sample)
    denominator = comb(population, sample)

    return numerator / denominator

```

Application: AI opponent uses this to predict when to save counters

3. Minimax with Alpha-Beta Pruning for AI

```
python
```

```
class AstraGameAI:
```

```
    """
```

```
    Minimax algorithm with alpha-beta pruning for optimal AI play
```

```
    Evaluates game tree to find best move
```

```
    """
```

```
def __init__(self, search_depth=4):
```

```
    self.search_depth = search_depth
```

```
    self.transposition_table = {} # Memoization for repeated states
```

```
def find_best_move(self, game_state):
```

```
    """
```

```
    Main entry point: find best move for current player
```

```
    """
```

```
    best_score = float('-inf')
```

```
    best_move = None
```

```
    alpha = float('-inf')
```

```
    beta = float('inf')
```

```
    for possible_move in game_state.get_legal_moves():
```

```
        # Make move
```

```
        new_state = game_state.make_move(possible_move)
```

```
        # Evaluate move
```

```
        score = self.minimax(new_state, self.search_depth - 1, alpha, beta, False)
```

```
        # Undo move (implicit in new_state approach)
```

```
        if score > best_score:
```

```
            best_score = score
```

```
            best_move = possible_move
```

```
    alpha = max(alpha, score)
```

```
    return best_move
```

```
def minimax(self, state, depth, alpha, beta, maximizing_player):
```

```
    """
```

```
    Recursive minimax with alpha-beta pruning
```

```
    """
```

```
    # Check transposition table
```

```
    state_hash = hash(state)
```

```
    if state_hash in self.transposition_table:
```

```
return self.transposition_table[state_hash]
```

```
# Terminal conditions
```

```
if depth == 0 or state.is_game_over():  
    score = self.evaluate_position(state)  
    self.transposition_table[state_hash] = score  
    return score
```

```
if maximizing_player:
```

```
    max_eval = float('-inf')  
    for move in state.get_legal_moves():  
        new_state = state.make_move(move)  
        eval = self.minimax(new_state, depth - 1, alpha, beta, False)  
        max_eval = max(max_eval, eval)  
        alpha = max(alpha, eval)  
        if beta <= alpha:  
            break # Beta cutoff
```

```
    self.transposition_table[state_hash] = max_eval  
    return max_eval
```

```
else:
```

```
    min_eval = float('inf')  
    for move in state.get_legal_moves():  
        new_state = state.make_move(move)  
        eval = self.minimax(new_state, depth - 1, alpha, beta, True)  
        min_eval = min(min_eval, eval)  
        beta = min(beta, eval)  
        if beta <= alpha:  
            break # Alpha cutoff
```

```
    self.transposition_table[state_hash] = min_eval  
    return min_eval
```

```
def evaluate_position(self, state):
```

```
    """
```

```
    Heuristic evaluation function
```

```
    Returns score from current player's perspective
```

```
    """
```

```
    score = 0
```

```
# Material (Life Points)
```

```
    score += (state.my_life - state.opponent_life) * 10
```

```
# Mantra advantage
```

```
score += (state.my_mantra - state.opponent_mantra) * 5

# Card advantage
score += (len(state.my_hand) - len(state.opponent_hand)) * 3

# Board control (active weapons)
score += (state.my_active_weapons - state.opponent_active_weapons) * 4

# Tempo (who's ahead in game)
if state.my_life < 10 and state.opponent_life > 15:
    score -= 20 # Losing badly

# Counter availability
counters_available = self.count_counters(state.my_hand, state.opponent_board)
score += counters_available * 2

return score
```

Complexity Analysis:

- **Without pruning:** $O(b^d)$ where b = branching factor (~ 10 moves/turn), d = depth
 - **With alpha-beta:** $O(b^{(d/2)})$ in best case - effectively doubles search depth!
-

4. Monte Carlo Tree Search (MCTS) Alternative

python

```
import random
```

```
import math
```

```
class MCTSNode:
```

```
    def __init__(self, state, parent=None, move=None):
```

```
        self.state = state
```

```
        self.parent = parent
```

```
        self.move = move
```

```
        self.children = []
```

```
        self.wins = 0
```

```
        self.visits = 0
```

```
        self.untried_moves = state.get_legal_moves()
```

```
    def uct_select_child(self):
```

```
        """
```

```
        UCT (Upper Confidence Bound for Trees) selection
```

```
        Balances exploitation vs exploration
```

```
        """
```

```
        log_parent_visits = math.log(self.parent.visits)
```

```
        return max(
```

```
            self.children,
```

```
            key=lambda c: c.wins/c.visits + math.sqrt(2 * log_parent_visits / c.visits)
```

```
        )
```

```
    def add_child(self, move, state):
```

```
        child = MCTSNode(state, parent=self, move=move)
```

```
        self.untried_moves.remove(move)
```

```
        self.children.append(child)
```

```
        return child
```

```
    def update(self, result):
```

```
        self.visits += 1
```

```
        self.wins += result
```

```
class MCTS_AI:
```

```
    """
```

```
    Monte Carlo Tree Search for game AI
```

```
    Better for complex games with high branching factor
```

```
    """
```

```
    def __init__(self, computation_budget=1000):
```

```
        self.computation_budget = computation_budget
```

```

def find_best_move(self, root_state):
    root = MCTSNode(root_state)

    for _ in range(self.computation_budget):
        node = root
        state = root_state.clone()

        # Selection - descend tree using UCT
        while node.untried_moves == [] and node.children != []:
            node = node.uct_select_child()
            state.make_move(node.move)

        # Expansion - add new child if possible
        if node.untried_moves != []:
            move = random.choice(node.untried_moves)
            state.make_move(move)
            node = node.add_child(move, state)

        # Simulation - play out random game
        while not state.is_game_over():
            move = random.choice(state.get_legal_moves())
            state.make_move(move)

        # Backpropagation - update all nodes in path
        result = state.get_result() # 1 for win, 0 for loss
        while node is not None:
            node.update(result)
            result = 1 - result # Flip for opponent
            node = node.parent

        # Return most visited child (most explored = most promising)
    return max(root.children, key=lambda c: c.visits).move

```

Advantages over Minimax:

- Handles uncertainty better
- Anytime algorithm (can stop early)
- Self-improving through simulation
- Better for high branching factor

CHAPTER 8: PYTHON PACKAGE DEVELOPMENT

Package Structure

```

astra-game/
├── astra/
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── weapon.py
│   │   ├── player.py
│   │   ├── game_state.py
│   │   └── elements.py
│   ├── ai/
│   │   ├── __init__.py
│   │   ├── minimax.py
│   │   ├── mcts.py
│   │   └── heuristics.py
│   ├── algorithms/
│   │   ├── __init__.py
│   │   ├── optimal_play.py
│   │   ├── deck_builder.py
│   │   └── counter_system.py
│   ├── data/
│   │   ├── weapons.json
│   │   ├── mythology.json
│   │   └── balance_data.json
│   └── utils/
│       ├── __init__.py
│       ├── validators.py
│       └── formatters.py
├── tests/
│   ├── test_weapons.py
│   ├── test_combat.py
│   └── test_ai.py
├── examples/
│   ├── basic_game.py
│   ├── ai_battle.py
│   └── deck_analysis.py
├── docs/
└── api.md

```



```
| |— mythology_guide.md
| |— algorithm_analysis.md
|— setup.py
|— requirements.txt
|— README.md
|— LICENSE
```

Core Package Code

```
python
```

```
# astra/core/weapon.py
```

```
from dataclasses import dataclass
from typing import List, Dict, Optional
from enum import Enum
```

```
class Element(Enum):
    FIRE = "fire"
    WATER = "water"
    WIND = "wind"
    EARTH = "earth"
    THUNDER = "thunder"
    SERPENT = "serpent"
    GARUDA = "garuda"
    DIVINE = "divine"
```

```
class Tier(Enum):
    CONVENTIONAL = 1
    ELEMENTAL = 2
    CELESTIAL = 3
    SUPREME = 4
```

```
class Rarity(Enum):
    COMMON = 1
    RARE = 2
    EPIC = 3
    LEGENDARY = 4
    MYTHIC = 5
```

```
@dataclass
```

```
class StatusEffect:
    name: str
    duration: int
    damage_per_turn: int = 0
    effect_type: str = "" # burn, freeze, stun, poison
```

```
@dataclass
```

```
class Weapon:
    """
    Base weapon class representing any weapon card in ASTRA
    """
    id: str
    name: str
```

```

power: int
mantra_cost: int
element: Element
tier: Tier
rarity: Rarity
counters: List[Element]
countered_by: List[Element]
status_effects: List[StatusEffect]
special_abilities: Dict[str, any]
mythology_text: str
lore_description: str

def can_counter(self, other_weapon: 'Weapon') -> bool:
    """Check if this weapon counters another weapon"""
    return other_weapon.element in self.counters

def is_countered_by(self, other_weapon: 'Weapon') -> bool:
    """Check if this weapon is countered by another"""
    return other_weapon.element in self.countered_by

def calculate_damage(self, target_life: int, game_state: 'GameState') -> int:
    """Calculate actual damage dealt considering game state"""
    base_damage = self.power

    # Check for special abilities
    if 'lethal_bonus' in self.special_abilities:
        if target_life <= self.power:
            base_damage *= self.special_abilities['lethal_bonus']

    if 'pierce' in self.special_abilities:
        # Pierce ignores defense
        return base_damage

    return base_damage

def __repr__(self):
    return f'Weapon({self.name}, Power:{self.power}, Cost:{self.mantra_cost})'

```

python

```
# astra/core/player.py
```

```
from typing import List, Optional
from .weapon import Weapon
from .game_state import GameState
```

```
class Player:
```

```
    """Represents a player in the game"""
```

```
    def __init__(self, name: str, deck: List[Weapon]):
```

```
        self.name = name
```

```
        self.deck = deck.copy()
```

```
        self.hand: List[Weapon] = []
```

```
        self.discard_pile: List[Weapon] = []
```

```
        self.active_weapons: List[Weapon] = []
```

```
        self.life_points = 20
```

```
        self.mantra_energy = 3
```

```
        self.max_mantra = 10
```

```
        self.status_effects: List[StatusEffect] = []
```

```
    # Shuffle deck
```

```
    import random
```

```
    random.shuffle(self.deck)
```

```
    # Draw starting hand
```

```
    self.draw_cards(5)
```

```
    def draw_cards(self, count: int = 1):
```

```
        """Draw cards from deck to hand"""
```

```
        for _ in range(count):
```

```
            if self.deck:
```

```
                card = self.deck.pop(0)
```

```
                self.hand.append(card)
```

```
            else:
```

```
                # Deck empty - could reshuffle discard or lose
```

```
                pass
```

```
    def play_weapon(self, weapon: Weapon, game_state: GameState) -> bool:
```

```
        """
```

```
        Attempt to play a weapon from hand
```

```
        Returns True if successful, False if invalid play
```

```

"""
if weapon not in self.hand:
    return False

if weapon.mantra_cost > self.mantra_energy:
    return False

# Pay cost
self.mantra_energy -= weapon.mantra_cost

# Move from hand to active
self.hand.remove(weapon)
self.active_weapons.append(weapon)

# Trigger special abilities
self.trigger_on_play_effects(weapon, game_state)

return True

def take_damage(self, damage: int):
    """Reduce life points"""
    self.life_points = max(0, self.life_points - damage)

def gain_life(self, amount: int):
    """Increase life points"""
    self.life_points += amount

def gain_mantra(self, amount: int):
    """Increase mantra energy"""
    self.mantra_energy = min(self.max_mantra, self.mantra_energy + amount)

def apply_status_effects(self):
    """Apply all active status effects"""
    for effect in self.status_effects:
        if effect.effect_type == 'burn' or effect.effect_type == 'poison':
            self.take_damage(effect.damage_per_turn)

        effect.duration -= 1

# Remove expired effects
self.status_effects = [e for e in self.status_effects if e.duration > 0]

def is_defeated(self) -> bool:

```

```
"""Check if player has lost"""
```

```
return self.life_points <= 0
```

```
python
```

```
# astra/core/game_state.py
```

```
from typing import List, Optional, Tuple
from .player import Player
from .weapon import Weapon
```

```
class GameState:
```

```
    """Represents the complete state of a game"""
```

```
    def __init__(self, player1: Player, player2: Player):
```

```
        self.player1 = player1
```

```
        self.player2 = player2
```

```
        self.current_player = player1
```

```
        self.other_player = player2
```

```
        self.turn_number = 1
```

```
        self.phase = "mantra" # mantra, draw, main, attack, defend, damage, end
```

```
        self.game_over = False
```

```
        self.winner = None
```

```
        self.attack_declared = None
```

```
        self.defense_declared = None
```

```
    def start_turn(self):
```

```
        """Begin a new turn"""
```

```
        # Mantra phase
```

```
        self.current_player.gain_mantra(1)
```

```
        self.phase = "draw"
```

```
        # Draw phase
```

```
        self.current_player.draw_cards(1)
```

```
        self.phase = "main"
```

```
        # Apply status effects
```

```
        self.current_player.apply_status_effects()
```

```
    def declare_attack(self, weapon: Weapon, target: Player):
```

```
        """Current player declares an attack"""
```

```
        if self.phase != "attack":
```

```
            return False
```

```
        self.attack_declared = (weapon, target)
```

```
        self.phase = "defend"
```

```
        return True
```

```

def declare_defense(self, weapon: Optional[Weapon]):
    """Other player declares defense (or passes)"""
    if self.phase != "defend":
        return False

    self.defense_declared = weapon
    self.phase = "damage"
    self.resolve_combat()
    return True

def resolve_combat(self):
    """Resolve attack and defense"""
    if not self.attack_declared:
        return

    attacking_weapon, target = self.attack_declared
    defending_weapon = self.defense_declared

    if defending_weapon is None:
        # No defense - direct damage
        damage = attacking_weapon.calculate_damage(target.life_points, self)
        target.take_damage(damage)

        # Apply status effects
        for effect in attacking_weapon.status_effects:
            target.status_effects.append(effect)

    else:
        # Resolve counter system
        if defending_weapon.can_counter(attacking_weapon):
            # Defender counters successfully
            attacker_damage = 0
            defender_damage = defending_weapon.power

            self.current_player.take_damage(defender_damage)

        elif attacking_weapon.element == defending_weapon.element:
            # Same element
            if attacking_weapon.tier == Tier.SUPREME and defending_weapon.tier == Tier.SUPREME:
                # Mutual destruction, no damage
                attacker_damage = 0
                defender_damage = 0

```



```
# Both weapons destroyed
```

```
self.current_player.active_weapons.remove(attacking_weapon)
```

```
self.other_player.active_weapons.remove(defending_weapon)
```

```
self.current_player.discard_pile.append(attacking_weapon)
```

```
self.other_player.discard_pile.append(defending_weapon)
```

```
else:
```

```
# Both deal damage
```

```
attacker_damage = attacking_weapon.power
```

```
defender_damage = defending_weapon.power
```

```
target.take_damage(attacker_damage)
```

```
self.current_player.take_damage(defender_damage)
```

```
else:
```

```
# Attacker wins
```

```
attacker_damage = attacking_weapon.power
```

```
defender_damage = 0
```

```
target.take_damage(attacker_damage)
```

```
# Apply status effects
```

```
for effect in attacking_weapon.status_effects:
```

```
    target.status_effects.append(effect)
```

```
# Clear declarations
```

```
self.attack_declared = None
```

```
self.defense_declared = None
```

```
self.phase = "end"
```

```
def end_turn(self):
```

```
    """End current turn and switch players"""
```

```
# Switch players
```

```
self.current_player, self.other_player = self.other_player, self.current_player
```

```
self.turn_number += 1
```

```
self.phase = "mantra"
```

```
# Check win condition
```

```
if self.player1.is_defeated():
```

```
    self.game_over = True
```

```
    self.winner = self.player2
```

```
elif self.player2.is_defeated():
```

```
    self.game_over = True
```

```
    self.winner = self.player1
```

```
def get_legal_moves(self) -> List:
    """Return all legal moves in current state"""
    if self.phase == "main":
        # Can play weapons from hand
        playable = [w for w in self.current_player.hand
                     if w.mantra_cost <= self.current_player.mantra_energy]
        return playable

    elif self.phase == "attack":
        # Can attack with active weapons
        return self.current_player.active_weapons

    elif self.phase == "defend":
        # Can defend with weapons in hand
        return self.other_player.hand + [None] # None = don't defend

    return []

def clone(self):
    """Create deep copy of game state for AI simulation"""
    import copy
    return copy.deepcopy(self)
```

Installation and Usage

python

```
# setup.py
```

```
from setuptools import setup, find_packages
```

```
setup(
    name="astra-game",
    version="1.0.0",
    description="ASTRA: Weapons of the Gods - Strategic Card Game Engine",
    author="Your Name",
    author_email="your.email@example.com",
    packages=find_packages(),
    install_requires=[
        "numpy>=1.20.0",
        "dataclasses-json>=0.5.0",
        "typing-extensions>=3.10.0",
    ],
    extras_require={
        "dev": [
            "pytest>=6.0.0",
            "pytest-cov>=2.12.0",
            "black>=21.0.0",
            "flake8>=3.9.0",
        ],
        "ai": [
            "scipy>=1.7.0",
            "scikit-learn>=0.24.0",
        ]
    },
    classifiers=[
        "Development Status :: 4 - Beta",
        "Intended Audience :: Developers",
        "Topic :: Games/Entertainment :: Board Games",
        "Programming Language :: Python :: 3.8",
        "Programming Language :: Python :: 3.9",
        "Programming Language :: Python :: 3.10",
    ],
    python_requires=">=3.8",
)
```

```
python
```

```
# Example usage: examples/basic_game.py
```

```
from astra.core.weapon import Weapon, Element, Tier, Rarity
from astra.core.player import Player
from astra.core.game_state import GameState
```

```
# Create some weapons
```

```
agneyastra = Weapon(
    id="001",
    name="Agneyastra",
    power=7,
    mantra_cost=3,
    element=Element.FIRE,
    tier=Tier.ELEMENTAL,
    rarity=Rarity.EPIC,
    counters=[Element.WIND],
    countered_by=[Element.WATER],
    status_effects=[],
    special_abilities={},
    mythology_text="Agni's fire weapon",
    lore_description="Weapon of the fire god that burns everything"
)
```

```
varunastra = Weapon(
    id="002",
    name="Varunastra",
    power=7,
    mantra_cost=3,
    element=Element.WATER,
    tier=Tier.ELEMENTAL,
    rarity=Rarity.EPIC,
    counters=[Element.FIRE],
    countered_by=[Element.WIND],
    status_effects=[],
    special_abilities={},
    mythology_text="Varuna's water weapon",
    lore_description="Creates devastating floods"
)
```

```
# Create decks (simplified - normally load from JSON)
```

```
deck1 = [agneyastra] * 20 + [varunastra] * 20
deck2 = [varunastra] * 20 + [agneyastra] * 20
```

```

# Create players
player1 = Player("Arjuna", deck1)
player2 = Player("Karna", deck2)

# Create game
game = GameState(player1, player2)

# Play game
while not game.game_over:
    game.start_turn()

    # Simple AI: play first playable card
    legal_moves = game.get_legal_moves()
    if legal_moves and game.phase == "main":
        weapon = legal_moves[0]
        game.current_player.play_weapon(weapon, game)

    game.end_turn()

print(f"Winner: {game.winner.name}")

```


CHAPTER 9: COMPUTER SCIENCE APPLICATIONS

Novel Algorithms Inspired by ASTRA

1. Counter-Chain Optimization Problem

Problem Definition:

Given a set of N items where each item i :

- Has value V_i
- Has cost C_i
- Counters set S_i (items it defeats)
- Countered-by set T_i (items that defeat it)

Find optimal sequence of items to maximize total value
while minimizing counter vulnerability

This is NP-hard (reduction from Set Cover problem)

Novel Algorithm: Dynamic Counter-Aware Selection

python

```
class CounterChainOptimizer:
```

```
    """
```

```
    Novel algorithm for counter-aware optimal play
```

```
    Applicable to: adversarial systems, security, game theory
```

```
    """
```

```
def optimize_play_sequence(self, available_items, opponent_model, budget):
```

```
    """
```

```
    Uses reinforcement learning + graph theory
```

```
    Innovation: Models counter relationships as directed graph  
    with edge weights representing counter effectiveness
```

```
    """
```

```
    # Build counter relationship graph
```

```
    G = self.build_counter_graph(available_items)
```

```
    # Use modified Bellman-Ford to find optimal paths
```

```
    # considering opponent's likely responses
```

```
    optimal_sequence = []
```

```
    remaining_budget = budget
```

```
for round in range(self.max_rounds):
```

```
    # Calculate expected value of each item
```

```
    # considering opponent's counter probability
```

```
    item_scores = {}
```

```
for item in available_items:
```

```
    if item.cost > remaining_budget:
```

```
        continue
```

```
    # Base value
```

```
    expected_value = item.value
```

```
    # Discount by counter probability
```

```
    counter_prob = self.calculate_counter_probability(  
        item, opponent_model, G  
    )
```

```
    expected_value *= (1 - counter_prob)
```

```
    # Bonus for countering opponent's likely plays
```

```
    counter_bonus = self.calculate_counter_bonus(  
        item, opponent_model, G  
    )
```

```
expected_value += counter_bonus
```

```
# Future value (can this enable future plays?)
```

```
synergy_value = self.calculate_synergy(  
    item, optimal_sequence, available_items  
)
```

```
expected_value += synergy_value
```

```
item_scores[item] = expected_value / item.cost # Value per cost
```

```
# Select best item
```

```
if not item_scores:
```

```
    break
```

```
best_item = max(item_scores, key=item_scores.get)
```

```
optimal_sequence.append(best_item)
```

```
remaining_budget -= best_item.cost
```

```
available_items.remove(best_item)
```

```
# Update opponent model (they adapt to our plays)
```

```
opponent_model.update(best_item)
```

```
return optimal_sequence
```

```
def build_counter_graph(self, items):
```

```
    """
```

```
    Build directed graph  $G = (V, E)$  where:
```

```
     $V = \text{items}$ 
```

```
     $E = \text{counter relationships with weights}$ 
```

```
    """
```

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
for item in items:
```

```
    G.add_node(item.id, data=item)
```

```
for item_i in items:
```

```
    for item_j in items:
```

```
        if item_i.id == item_j.id:
```

```
            continue
```

```
# Add edge if item_i counters item_j
```

```
if item_j.element in item_i.counters:
```



```
# Weight = effectiveness of counter
weight = item_i.power / max(item_j.power, 1)
G.add_edge(item_i.id, item_j.id, weight=weight)
```

```
return G
```

```
def calculate_counter_probability(self, item, opponent_model, graph):
    """
    Estimate probability opponent has counter to this item
    Uses Bayesian inference + graph analysis
    """
    counter_items = [n for n in graph.nodes()
                     if graph.has_edge(n, item.id)]

    total_prob = 0
    for counter_id in counter_items:
        # Probability opponent has this counter in hand/deck
        prob_has_counter = opponent_model.get_probability(counter_id)

        # Probability they'll actually play it
        prob_will_play = opponent_model.get_play_probability(
            counter_id, item
        )

        total_prob += prob_has_counter * prob_will_play

    return min(total_prob, 1.0)
```

Applications Beyond Gaming:

- **Cybersecurity:** Attack/defense selection in adversarial networks
- **Military Strategy:** Weapon system selection considering countermeasures
- **Economics:** Investment selection considering market counters
- **Debate/Argumentation:** Argument selection considering counter-arguments

2. Mythological Knowledge Graph Algorithm

```
python
```

```
class MythologyKnowledgeGraph:
```

```
    """
```

Novel: Directed Acyclic Graph (DAG) representing mythological relationships

Innovation: Combines ontology reasoning with graph traversal for story generation and consistency checking

```
    """
```

```
def __init__(self):
```

```
    import networkx as nx
```

```
    self.graph = nx.MultiDiGraph()
```

```
    self.entity_types = ['deity', 'weapon', 'event', 'location', 'concept']
```

```
def add_mythological_fact(self, subject, relation, object, source_text):
```

```
    """
```

Add fact to knowledge graph with provenance tracking

Example:

```
add_fact('Brahmastra', 'created_by', 'Brahma', 'Mahabharata Ch.7')
```

```
    """
```

```
self.graph.add_edge(
```

```
    subject,
```

```
    object,
```

```
    relation=relation,
```

```
    source=source_text,
```

```
    timestamp=self.get_timestamp()
```

```
)
```

```
def query_relationships(self, entity, relation_type=None, depth=1):
```

```
    """
```

Query all relationships for an entity up to certain depth

Example: query_relationships('Arjuna', 'obtained_weapon', depth=2)

Returns: All weapons Arjuna obtained and their properties

```
    """
```

```
import networkx as nx
```

```
results = []
```

```
if depth == 1:
```

```
    # Direct relationships
```

```
    for successor in self.graph.successors(entity):
```

```
        edges = self.graph.get_edge_data(entity, successor)
```

```

for key, edge_data in edges.items():
    if relation_type is None or edge_data['relation'] == relation_type:
        results.append({
            'subject': entity,
            'relation': edge_data['relation'],
            'object': successor,
            'source': edge_data['source']
        })

```

else:

Multi-hop relationships using BFS

visited = set()

queue = [(entity, 0, [])]

while queue:

current, current_depth, path = queue.pop(0)

if current_depth >= depth:

continue

if current in visited:

continue

visited.add(current)

for successor in self.graph.successors(current):

edges = self.graph.get_edge_data(current, successor)

for key, edge_data in edges.items():

new_path = path + [(current, edge_data['relation'], successor)]

if current_depth + 1 == depth:

```

results.append({
    'path': new_path,
    'depth': depth
})

```

else:

queue.append((successor, current_depth + 1, new_path))

return results

def verify_consistency(self, new_fact):

"""

Check if new fact contradicts existing knowledge

Uses logical inference rules specific to mythology

"""

```
subject, relation, obj = new_fact
```

```
# Check for contradictions
```

```
contradictions = []
```

```
# Example rule: A weapon can't be created by multiple deities
```

```
if relation == 'created_by':
```

```
    existing_creators = self.query_relationships(  
        subject, 'created_by', depth=1  
    )
```

```
    if existing_creators and obj not in [r['object'] for r in existing_creators]:
```

```
        contradictions.append({  
            'type': 'multiple_creators',  
            'existing': existing_creators,  
            'new': new_fact  
        })
```

```
# Rule: Circular defeat relationships are impossible
```

```
if relation == 'defeats':
```

```
    # Check if object defeats subject (directly or indirectly)
```

```
    paths = self.find_all_paths(obj, subject, relation='defeats')
```

```
    if paths:
```

```
        contradictions.append({  
            'type': 'circular_defeat',  
            'path': paths[0]  
        })
```

```
return contradictions
```

```
def generate_story_template(self, start_entity, end_entity):
```

```
    """
```

```
    Generate story connecting two entities using graph paths
```

```
    Novel application: Automated story generation from knowledge graph
```

```
    """
```

```
    paths = self.find_all_paths(start_entity, end_entity)
```

```
    if not paths:
```

```
        return None
```

```
# Select most interesting path (heuristic: most relationships)
```

```
best_path = max(paths, key=lambda p: len(p))
```

```
# Convert path to narrative template
```

```
story_template = []
```

```
for i in range(len(best_path) - 1):
```

```
    current = best_path[i]
```

```
    next_entity = best_path[i + 1]
```

```
    edges = self.graph.get_edge_data(current, next_entity)
```

```
    relation = edges[0]['relation']
```

```
# Template based on relation type
```

```
templates = {
```

```
    'obtained_weapon': f'{current} obtained the divine weapon {next_entity}',
```

```
    'used_against': f'{current} used {next_entity} in battle',
```

```
    'defeated': f'{current} was defeated by {next_entity}',
```

```
    'created_by': f'{current} was created by {next_entity}',
```

```
}
```

```
story_template.append(
```

```
    templates.get(relation, f'{current} {relation} {next_entity}')
```

```
)
```

```
return ". ".join(story_template) + "."
```

```
def find_all_paths(self, start, end, relation=None, max_length=5):
```

```
    """Find all paths between two entities"""
```

```
    import networkx as nx
```

```
    try:
```

```
        all_paths = list(nx.all_simple_paths(
```

```
            self.graph, start, end, cutoff=max_length
```

```
        ))
```

```
    if relation:
```

```
        # Filter by relation type
```

```
        filtered_paths = []
```

```
        for path in all_paths:
```

```
            valid = True
```

```
            for i in range(len(path) - 1):
```

```
                edges = self.graph.get_edge_data(path[i], path[i+1])
```

```
                if not any(e['relation'] == relation for e in edges.values()):
```

```
                    valid = False
```

```
                    break
```

```
            if valid:
```

```
        filtered_paths.append(path)
    return filtered_paths

return all_paths
except nx.NetworkXNoPath:
    return []
```

Research Applications:

- **Digital Humanities:** Analyzing epic literature patterns
 - **Ontology Engineering:** Building cultural knowledge bases
 - **AI Story Generation:** Creating narratives from structured data
 - **Education:** Interactive mythology learning systems
-

3. Adaptive Deck Building Algorithm (Genetic Algorithm)

```
python
```

```
import random
import numpy as np
```

```
class GeneticDeckBuilder:
```

```
    """
```

Uses genetic algorithms to evolve optimal deck compositions

Innovation: Fitness function based on win rate against meta decks

Mutation considers mythological constraints (lore-friendly combinations)

```
    """
```

```
def __init__(self, card_pool, population_size=100, generations=50):
```

```
    self.card_pool = card_pool
    self.population_size = population_size
    self.generations = generations
    self.deck_size = 40
```

```
def evolve_optimal_deck(self, meta_decks, constraints=None):
```

```
    """
```

Evolve deck that performs best against meta

Returns: Optimal deck composition

```
    """
```

Initialize random population

```
population = self.initialize_population()
```

```
for generation in range(self.generations):
```

```
    # Evaluate fitness
```

```
    fitness_scores = [
        self.evaluate_fitness(deck, meta_decks)
        for deck in population
    ]
```

```
    # Selection (tournament selection)
```

```
    selected = self.tournament_selection(population, fitness_scores)
```

```
    # Crossover (combine decks)
```

```
    offspring = []
    for i in range(0, len(selected), 2):
        if i + 1 < len(selected):
            child1, child2 = self.crossover(selected[i], selected[i+1])
            offspring.extend([child1, child2])
```

Mutation (random changes)

for deck in offspring:

if random.random() < 0.3: *# 30% mutation rate*

self.mutate(deck, constraints)

Elitism: keep best decks from previous generation

population = self.select_elite(population, fitness_scores, count=10)

population.extend(offspring[:self.population_size - 10])

Progress report

best_fitness = max(fitness_scores)

print(f'Generation {generation}: Best fitness = {best_fitness:.2f}')

Return best deck

final_fitness = [self.evaluate_fitness(d, meta_decks) for d in population]

best_deck = population[np.argmax(final_fitness)]

return best_deck

def initialize_population(self):

"""Create initial random population"""

population = []

for _ in range(self.population_size):

deck = random.choices(self.card_pool, k=self.deck_size)

population.append(deck)

return population

def evaluate_fitness(self, deck, meta_decks):

"""

Simulate games against meta decks

Fitness = win rate + deck coherence score

"""

from .ai.minimax import AstraGameAI

wins = 0

total_games = len(meta_decks) * 5 *# 5 games per matchup*

for meta_deck in meta_decks:

for _ in range(5):

Simulate game

winner = self.simulate_game(deck, meta_deck)

if winner == 'deck':


```
wins += 1
```

```
win_rate = wins / total_games
```

```
# Coherence score (bonus for synergistic decks)
```

```
coherence = self.calculate_deck_coherence(deck)
```

```
# Mana curve score (bonus for good distribution)
```

```
curve_score = self.calculate_curve_score(deck)
```

```
# Mythological theme bonus (lore-friendly combinations)
```

```
theme_score = self.calculate_theme_score(deck)
```

```
fitness = (
```

```
    win_rate * 100 +      # Primary: win rate
```

```
    coherence * 10 +      # Secondary: synergy
```

```
    curve_score * 5 +     # Tertiary: curve
```

```
    theme_score * 3       # Bonus: theme
```

```
)
```

```
return fitness
```

```
def calculate_deck_coherence(self, deck):
```

```
    """
```

```
    Measure how well cards work together
```

```
    Higher score for elemental focus, synergies
```

```
    """
```

```
    element_counts = {}
```

```
    for card in deck:
```

```
        element_counts[card.element] = element_counts.get(card.element, 0) + 1
```

```
# Reward focused decks (2-3 main elements)
```

```
primary_elements = sorted(element_counts.values(), reverse=True)[:3]
```

```
focus_score = sum(primary_elements) / len(deck)
```

```
# Reward synergies
```

```
synergy_score = 0
```

```
for i, card1 in enumerate(deck):
```

```
    for card2 in deck[i+1:]:
```

```
        if self.have_synergy(card1, card2):
```

```
            synergy_score += 1
```

```
return focus_score + (synergy_score / 100)
```

```

def calculate_curve_score(self, deck):
    """
    Evaluate mana curve quality
    Ideal: many low-cost, some mid, few high
    """
    cost_distribution = [0] * 7 # Costs 0-6+

    for card in deck:
        cost = min(card.mantra_cost, 6)
        cost_distribution[cost] += 1

    # Ideal distribution (based on expert decks)
    ideal = [0, 8, 10, 8, 6, 4, 4] # 0-cost, 1-cost, 2-cost, etc.

    # Calculate distance from ideal (lower is better)
    distance = sum(abs(cost_distribution[i] - ideal[i]) for i in range(7))

    # Convert to score (max 1.0)
    max_distance = sum(ideal)
    curve_score = 1.0 - (distance / max_distance)

    return curve_score

def calculate_theme_score(self, deck):
    """
    Bonus for mythologically coherent themes
    Example: All weapons from same epic, or same deity
    """
    themes = {
        'mahabharata': 0,
        'ramayana': 0,
        'shiva': 0,
        'vishnu': 0,
        'fire': 0,
        'water': 0
    }

    for card in deck:
        # Check card's mythological associations
        if hasattr(card, 'epic') and card.epic == 'mahabharata':
            themes['mahabharata'] += 1
        if hasattr(card, 'epic') and card.epic == 'ramayana':
            themes['ramayana'] += 1
        if hasattr(card, 'deity') and card.deity == 'shiva':

```

```

        themes['shiva'] += 1
    if card.element == 'FIRE':
        themes['fire'] += 1

# Reward strong themes (>50% of deck)
    strongest_theme = max(themes.values())
    if strongest_theme >= 20: # Half the deck
        return 1.0
    elif strongest_theme >= 15:
        return 0.5
    else:
        return 0.0

def crossover(self, parent1, parent2):
    """
    Combine two decks to create offspring
    Uses single-point crossover
    """
    crossover_point = random.randint(1, self.deck_size - 1)

    child1 = parent1[:crossover_point] + parent2[crossover_point:]
    child2 = parent2[:crossover_point] + parent1[crossover_point:]

    return child1, child2

def mutate(self, deck, constraints=None):
    """
    Randomly modify deck
    Respects constraints (e.g., max 3 copies of a card)
    """
    mutation_count = random.randint(1, 5)

    for _ in range(mutation_count):
        # Choose random position
        pos = random.randint(0, len(deck) - 1)

        # Replace with random card from pool
        new_card = random.choice(self.card_pool)

        # Check constraints
        if constraints:
            card_count = sum(1 for c in deck if c.id == new_card.id)
            if card_count >= 3: # Max 3 copies
                continue

```

```
deck[pos] = new_card
```

```
def tournament_selection(self, population, fitness_scores, tournament_size=5):  
    """Select parents using tournament selection"""  
    selected = []  
  
    for _ in range(len(population)):  
        # Random tournament  
        tournament = random.sample(list(zip(population, fitness_scores)), tournament_size)  
        winner = max(tournament, key=lambda x: x[1])[0]  
        selected.append(winner)  
  
    return selected  
  
def select_elite(self, population, fitness_scores, count):  
    """Keep best decks"""  
    sorted_pop = sorted(  
        zip(population, fitness_scores),  
        key=lambda x: x[1],  
        reverse=True  
    )  
  
    return [deck for deck, _ in sorted_pop[:count]]
```

Applications:

- **Automated Deck Building:** Players input meta, get optimal deck
- **Balance Testing:** Game designers find overpowered combinations
- **Meta Analysis:** Understand evolving strategies
- **AI Training:** Generate diverse training data

CHAPTER 10: ARTIFICIAL INTELLIGENCE INTEGRATION

Machine Learning for Card Balance

```
python
```

```

import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

class CardBalancePredictor:
    """
    ML model to predict card win rates and suggest balance changes

    Features: power, cost, element, rarity, abilities
    Target: win rate when included in deck
    """

    def __init__(self):
        self.model = RandomForestRegressor(
            n_estimators=100,
            max_depth=10,
            random_state=42
        )
        self.feature_names = [
            'power', 'cost', 'element_encoded', 'tier', 'rarity',
            'has_burn', 'has_poison', 'has_stun', 'has_pierce',
            'counter_count', 'countered_by_count'
        ]

    def extract_features(self, card):
        """Convert card to feature vector"""
        features = [
            card.power,
            card.mantra_cost,
            self.encode_element(card.element),
            card.tier.value,
            card.rarity.value,
            int('burn' in [e.effect_type for e in card.status_effects]),
            int('poison' in [e.effect_type for e in card.status_effects]),
            int('stun' in [e.effect_type for e in card.status_effects]),
            int('pierce' in card.special_abilities),
            len(card.counters),
            len(card.countered_by)
        ]

        return np.array(features)

    def train(self, cards_data, win_rates):

```

```
"""
```

Train model on historical card performance data

cards_data: List of Card objects

win_rates: List of win rates (0.0 to 1.0) for each card

```
"""
```

```
X = np.array([self.extract_features(card) for card in cards_data])
```

```
y = np.array(win_rates)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
self.model.fit(X_train, y_train)
```

```
# Evaluate
```

```
train_score = self.model.score(X_train, y_train)
```

```
test_score = self.model.score(X_test, y_test)
```

```
print(f"Train R2: {train_score:.3f}")
```

```
print(f"Test R2: {test_score:.3f}")
```

```
# Feature importance
```

```
importances = self.model.feature_importances_
```

```
for name, importance in zip(self.feature_names, importances):
```

```
    print(f"{name}: {importance:.3f}")
```

```
def predict_win_rate(self, card):
```

```
    """Predict win rate for a card"""
```

```
    features = self.extract_features(card).reshape(1, -1)
```

```
    predicted_win_rate = self.model.predict(features)[0]
```

```
    return predicted_win_rate
```

```
def suggest_balance_changes(self, card, target_win_rate=0.50):
```

```
    """
```

Suggest changes to make card balanced

Uses optimization to find minimal changes

```
    """
```

```
    current_win_rate = self.predict_win_rate(card)
```

```
    if abs(current_win_rate - target_win_rate) < 0.02:
```

```
        return "Card is balanced!"
```

```

suggestions = []

# Try adjusting power
test_card = card.copy()
for power_delta in [-2, -1, 1, 2]:
    test_card.power = card.power + power_delta
    new_win_rate = self.predict_win_rate(test_card)

    if abs(new_win_rate - target_win_rate) < abs(current_win_rate - target_win_rate):
        suggestions.append({
            'change': f'Power: {card.power} → {test_card.power}',
            'predicted_win_rate': new_win_rate,
            'improvement': abs(target_win_rate - new_win_rate)
        })

# Try adjusting cost
test_card = card.copy()
for cost_delta in [-1, 1]:
    new_cost = max(1, card.mantra_cost + cost_delta)
    test_card.mantra_cost = new_cost
    new_win_rate = self.predict_win_rate(test_card)

    if abs(new_win_rate - target_win_rate) < abs(current_win_rate - target_win_rate):
        suggestions.append({
            'change': f'Cost: {card.mantra_cost} → {new_cost}',
            'predicted_win_rate': new_win_rate,
            'improvement': abs(target_win_rate - new_win_rate)
        })

# Sort by improvement
suggestions.sort(key=lambda x: x['improvement'])

return suggestions[:3] # Top 3 suggestions

```

Neural Network for Play Prediction

python

```

import torch
import torch.nn as nn
import torch.optim as optim

class AstraPlayNet(nn.Module):
    """
    Neural network to predict optimal plays

    Input: Game state (encoded)
    Output: Probability distribution over possible moves
    """

    def __init__(self, state_size=256, hidden_size=512, num_actions=100):
        super(AstraPlayNet, self).__init__()

        self.network = nn.Sequential(
            nn.Linear(state_size, hidden_size),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(hidden_size, hidden_size // 2),
            nn.ReLU(),

            nn.Linear(hidden_size // 2, num_actions),
            nn.Softmax(dim=1)
        )

    def forward(self, state):
        return self.network(state)

class ReinforcementLearningAgent:
    """
    RL agent that learns to play ASTRA through self-play
    Uses Deep Q-Learning (DQN)
    """

    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size

```



```

self.memory = []
self.gamma = 0.95 # Discount factor
self.epsilon = 1.0 # Exploration rate
self.epsilon_min = 0.01
self.epsilon_decay = 0.995
self.learning_rate = 0.001

self.model = AstraPlayNet(state_size, 512, action_size)
self.optimizer = optim.Adam(self.model.parameters(), lr=self.learning_rate)
self.criterion = nn.MSELoss()

```

```

def remember(self, state, action, reward, next_state, done):
    """Store experience in replay memory"""
    self.memory.append((state, action, reward, next_state, done))
    if len(self.memory) > 10000:
        self.memory.pop(0)

```

```

def act(self, state, legal_actions):
    """
    Choose action using epsilon-greedy policy
    """
    if np.random.random() <= self.epsilon:
        # Explore: random action
        return random.choice(legal_actions)

    # Exploit: best predicted action
    state_tensor = torch.FloatTensor(state).unsqueeze(0)
    action_probs = self.model(state_tensor).detach().numpy()[0]

    # Mask illegal actions
    legal_mask = np.zeros(self.action_size)
    for action in legal_actions:
        legal_mask[action] = 1

    masked_probs = action_probs * legal_mask
    if masked_probs.sum() > 0:
        masked_probs /= masked_probs.sum()
        return np.argmax(masked_probs)
    else:
        return random.choice(legal_actions)

```

```

def replay(self, batch_size=32):
    """
    Train on random batch from memory

```

```

"""
if len(self.memory) < batch_size:
    return

minibatch = random.sample(self.memory, batch_size)

for state, action, reward, next_state, done in minibatch:
    target = reward
    if not done:
        next_state_tensor = torch.FloatTensor(next_state).unsqueeze(0)
        target = reward + self.gamma * torch.max(
            self.model(next_state_tensor)
        ).item()

    state_tensor = torch.FloatTensor(state).unsqueeze(0)
    target_f = self.model(state_tensor).detach().clone()
    target_f[0][action] = target

    # Train
    self.optimizer.zero_grad()
    output = self.model(state_tensor)
    loss = self.criterion(output, target_f)
    loss.backward()
    self.optimizer.step()

    # Decay exploration
    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

```


PART 3: MULTI-FORMAT VISION

CHAPTER 11: FORMAT 1 - Simple Card Game (UNO-Style)

Simplified ASTRA Rules

Objective: Be first to reduce opponent's Life Points to 0

Setup:

- Each player: 20 Life Points, 7 cards

- Draw pile in center
- Discard pile next to it

Turn Structure:

1. Draw 1 card
2. Play 1 card OR pass
3. Resolve effects

Card Types (Simplified):

ATTACK CARDS (60%):

AGNEYASTRA	
  	
DAMAGE: 7	
Countered by:	
WATER cards	

ACTION CARDS (30%):

- COUNTER: Block attack
- HEAL: Gain 3 Life
- DRAW 2: Draw 2 cards
- SKIP: Opponent skips turn
- REVERSE: Reverse turn order (multiplayer)

SPECIAL CARDS (10%):

- BRAHMASTRA: Deal 10 damage, can't be countered except by another Brahmastra
- DIVINE SHIELD: Block next 3 attacks

Simplified Counter System:

FIRE beats WIND
 WATER beats FIRE
 WIND beats WATER
 EARTH beats all (but rare)

Win Condition: Reduce opponent to 0 Life Points

UNO-Style Deck Composition (108 Cards)

FIRE ATTACKS (19 cards):
- # ASTRA: WEAPONS OF THE GODS
Complete Player & Developer Manual
The Ultimate Reference Guide for Players, Creators, and Developers

TABLE OF CONTENTS

PART 1: PLAYER'S JOURNEY GUIDE

- 1. [Beginner's Path](#beginners-path)
- 2. [Intermediate Mastery](#intermediate-mastery)
- 3. [Advanced Strategies](#advanced-strategies)
- 4. [Mythology Deep Dive](#mythology-deep-dive)
- 5. [Tournament Preparation](#tournament-prep)

PART 2: DEVELOPER'S TECHNICAL GUIDE

- 6. [Game Design Philosophy](#game-design)
- 7. [Algorithm Design & AI](#algorithms-ai)
- 8. [Python Package Development](#python-package)
- 9. [Computer Science Applications](#cs-applications)
- 10. [Artificial Intelligence Integration](#ai-integration)

PART 3: MULTI-FORMAT VISION

- 11. [Format 1: Simple Card Game (UNO-style)](#format-uno)
- 12. [Format 2: Clash Royale Style](#format-clash)
- 13. [Dhanurveda Weapons Integration](#dhanurveda)
- 14. [Monetization Models](#monetization)
- 15. [Creative Innovations](#innovations)

PART 4: CONTENT CREATION & BUSINESS

- 16. [Video Content Strategy](#video-strategy)
- 17. [Documentation Products](#documentation)
- 18. [Book Composition Ideas](#book-ideas)
- 19. [Manual Sales Strategy](#manual-sales)
- 20. [Long-term Vision](#long-term)

PART 1: PLAYER'S JOURNEY GUIDE

CHAPTER 1: BEGINNER'S PATH - Your First 7 Days

DAY 1: Understanding the Basics

Your First Game Setup

What You Need:

- 1 Starter Deck (40 cards pre-built)
- Life Point tracker (start at 20)
- Mantra Energy tracker (start at 3)
- Reference card showing counter system

First Turn Walkthrough

TURN 1 - You Go First:

1. Mantra Phase: You have 3 Mantra, gain +1 = 4 total
2. Draw Phase: Draw 1 card (now 6 cards in hand)
3. Main Phase: Play "Dhanusha" (costs 1 Mantra)
 - You now have 3 Mantra remaining
4. Attack Phase: Dhanusha attacks opponent (4 damage)
 - Opponent has no counter
 - Opponent loses 4 Life Points (now at 16)
5. End Phase: Your turn ends

Result: You dealt 4 damage, used 1 Mantra, have 5 cards left

Your First Counter

OPPONENT'S TURN:

1. They play "Agneyastra" (Fire, 7 power, costs 3)
2. They declare attack on you
3. YOU RESPOND: "I play Varunastra!" (Water, costs 3)
4. Resolution: Water counters Fire
 - Agneyastra deals 0 damage (countered)

- Varunastra deals 7 damage to opponent

5. Opponent loses 7 Life Points

Result: You successfully countered and dealt damage back!

DAY 2: Learning Elements

The Core Triangle

FIRE → beats → WIND WATER → beats → FIRE

WIND → beats → WATER EARTH → beats → THUNDER THUNDER → beats → WATER

Practice Exercise

Build a practice deck with:

- 10 Fire weapons
- 10 Water weapons
- 10 Wind weapons
- 10 Earth weapons

Play against yourself to see how counters work.

****Learning Goals:****

- Identify weapon elements quickly
- Know which element beats which
- Feel the rhythm of counter-play

DAY 3: Mantra Management

The Golden Rule

****"Never spend all your Mantra unless it wins the game"****

Mantra Curve Planning

Turn 1 (4 Mantra): Play 1-2 cost weapons Turn 2 (5 Mantra): Play 2-3 cost weapons

Turn 3 (6 Mantra): Play 3-4 cost weapons Turn 4 (7 Mantra): Save for counters OR play big weapon Turn 5+

(8+): Supreme weapons become playable

Exercise: The Mantra Challenge

Play a game where you track every Mantra spent:

- Did you waste any?
- Could you have saved Mantra for a counter?
- Did you miss opportunities to play more?

DAY 4: Reading Your Opponent

What to Watch

1. ****Elements They Play:**** If they play Fire twice, they probably have Fire deck
2. ****Mantra Count:**** If they have 6+ Mantra and pass, they're holding a counter
3. ****Hand Size:**** Many cards = many options, few cards = desperation
4. ****Life Points:**** Below 10? They might take risks. Above 15? They'll play safe.

Beginner Mind Tricks

- ****The Bluff:**** Pass your turn with 5 Mantra. Opponent thinks you have counter. They play defensively.
- ****The Bait:**** Play weak weapon first. They don't counter. Then play strong weapon.
- ****The All-In:**** When they're at 8 Life Points and you have Agneyastra (7 damage) + Agni Bana (5 damage), attack both!

DAY 5: Building Your First Deck

Starter Deck Template: "Fire Rush"

WEAPONS (25 cards):

- 4x Agni Bana (1 cost, 5 power) - Your bread and butter
- 3x Agneyastra (3 cost, 7 power) - Main damage
- 3x Surya Astra (3 cost, 7 power) - Backup fire
- 2x Dahana Astra (2 cost, 6 power) - Burn effect
- 3x Dhanusha (1 cost, 4 power) - Cheap early game
- 3x Khadga (2 cost, 5 power) - Filler
- 2x Gandiva (3 cost, 7 power) - Card draw
- 2x Tejas Kavach (2 cost, Defense) - Fire shield
- 3x Varunastra (3 cost, 7 power) - Anti-mirror

SUPPORT (15 cards):

- 4x Mantra Meditation - Gain 3 Mantra

- 3x Guru's Blessing - Draw 3 cards
- 2x Carmakara Shield - Block 4 damage
- 3x Healing cards - Gain life
- 2x Brahmastra (5 cost, 10 power) - Finisher
- 1x Sudarshana Chakra (4 cost, 7 power) - Pierce

****Why This Works:****

- Lots of cheap fire weapons for early pressure
- Burn effects stack up damage
- Varunastra stops mirror matches
- Support keeps you alive and drawing cards

DAY 6: Your First Victory

The Winning Mindset

1. ****Damage Counts:**** Keep mental math of total damage you can deal
2. ****Count Their Life:**** Always know how close they are to defeat
3. ****The Lethal Check:**** Before each turn, ask "Can I win this turn?"

Example Winning Play

GAME STATE:

- Opponent: 12 Life Points
- You: 15 Life Points
- Your Hand: Agneyastra (7), Surya Astra (7), Agni Bana (5)
- Your Mantra: 9
- Opponent's Mantra: 4

WINNING SEQUENCE:

1. Play Agneyastra (costs 3, you have 6 left) = 7 damage
2. They counter with water? NO - They only have 4 Mantra, can't afford Varunastra (costs 3) AND save Mantra for defense
3. Play Surya Astra (costs 3, you have 3 left) = 7 damage
4. Total damage = 14 damage
5. They have 12 Life Points
6. YOU WIN!

Even if they block one, the other gets through!

DAY 7: Review and Reflect

Self-Assessment Quiz

1. Can you name all 5 main elements? ✓
2. Can you explain one counter relationship? ✓
3. Have you won at least 1 game? ✓
4. Do you understand Mantra management? ✓
5. Can you build a 40-card deck? ✓

If you answered YES to all: You're ready for Intermediate!

CHAPTER 2: INTERMEDIATE MASTERY - Weeks 2-4

WEEK 2: Advanced Counter Play

The Counter Matrix

YOU PLAY FIRE:

Opponent Options:

- Counter with WATER = You take damage, lose weapon
- Counter with EARTH = Neutral, both deal damage
- Counter with nothing = You deal damage
- Counter with FIRE = Both weapons destroyed, no damage

YOUR RESPONSE:

- If they're likely to have water, don't play fire
- Bait with cheap fire first, see if they counter
- Then play expensive fire when they're out of water

Advanced Baiting Technique

TURN 3 SETUP:

1. You have: Agni Bana (5 power, 1 cost) and Agneyastra (7 power, 3 cost)
2. Play Agni Bana first
3. Opponent thinks: "Cheap weapon, not worth countering, save Mantra"
4. Agni Bana hits for 5 damage
5. NEXT TURN: Play Agneyastra
6. Opponent NOW counters, but you already got 5 damage through!

VS

BAD PLAY:

1. Play Agneyastra first
2. Opponent counters immediately
3. You dealt 0 damage, wasted 3 Mantra

WEEK 3: Deck Synergies

Building Synergy Chains

Example: Poison Chain Deck

CARD 1: Naagastra (6 power, poison 2 damage/turn)

CARD 2: Takshaka Shakti (7 power, uncurable poison)

CARD 3: Karkotaka Visha (6 power, curse -2 max HP)

CARD 4: Vasuki Pasha (6 power, bind - opponent can't play)

TURN SEQUENCE:

Turn 1: Naagastra - Opponent at 14 HP, poisoned

Turn 2: They take 2 poison (12 HP), you play Karkotaka (6 HP, max reduced to 18)

Turn 3: They take 2 poison (4 HP), you play Vasuki (bind them)

Turn 4: They can't play, take 2 poison (2 HP), you play any weapon = WIN

Total actual weapon damage: 18

Total poison damage: 6

Total victory: Turn 4

Synergy Categories

1. ****Element Synergy:**** All same element + boosters
2. ****Status Synergy:**** Stack burn/poison/freeze
3. ****Character Synergy:**** Arjuna + Gandiva + Krishna cards
4. ****Tier Synergy:**** Ramp to Supreme weapons
5. ****Counter Synergy:**** Disruption + control

WEEK 4: Meta Reading

What is "The Meta"?

The most popular strategies in your local game group or online.

****Example Local Metas:****

AGGRESSIVE META:

- Everyone plays fast fire/wind decks
- Games end by turn 5-6
- YOUR COUNTER: Play water control with healing (You survive early, win late)

CONTROL META:

- Everyone plays defensive water/earth
- Games go 10+ turns
- YOUR COUNTER: Play fast serpent poison (You don't need to attack, poison wins)

SUPREME META:

- Everyone ramps to Brahmastra/Pashupatastra
- Big weapon battles
- YOUR COUNTER: Play aggro, kill them before turn 7

Meta Deck Analysis Tool

For every deck you face, record:

- What element?
- How fast (aggro/midrange/control)?
- Win condition?
- Your win rate against it?

After 20 games, you'll see patterns!

CHAPTER 3: ADVANCED STRATEGIES - Month 2+

Psychological Warfare

Reading Tells

PHYSICAL TELLS:

- Quick play = Confident, probably has counter
- Hesitation = Calculating, might be close call
- Checking graveyard = Looking for specific counter
- Counting Mantra = Planning big play

GAMEPLAY TELLS:

- Passing with high Mantra = Holding counter
- Playing defensive card early = Weak hand
- Overextending attacks = No counters left
- Conservative play = Waiting for combo

Bluffing Techniques

The False Counter Bluff:

SETUP:

- You have 6 Mantra
- Opponent attacks with Agneyastra (7 power)
- You DON'T have Varunastra (water counter)

BLUFF:

1. Pause dramatically
2. Look at your hand intensely
3. Look at Mantra counter
4. Sigh and say "I'll take the damage"

RESULT:

- Opponent thinks you HAD counter but chose not to use it
- They think you're saving it for bigger threat
- Next turn they play conservatively
- You buy time to draw actual counter!

Tournament-Level Play

Swiss Round Strategy

ROUNDS 1-2: Test Your Deck

- Try different strategies
- Learn opponents' decks
- Don't reveal your best plays yet

ROUNDS 3-4: Adapt and Refine

- Adjust sideboard based on meta
- Start playing tighter
- Minimize mistakes

FINALS: Execute Perfectly

- Stick to proven strategy

- Don't experiment
- Play to win, not to impress

Sideboarding Guide

SIDEBOARD (10 cards):

Against Fire Decks:

- +3 Varunastra (water counter)
- +2 Water shields Remove:
- -3 Agneyastra (they'll counter)
- -2 Fire weapons

Against Control:

- +3 Poison weapons (win without attacking)
- +2 Card draw Remove:
- -3 Expensive weapons (too slow)
- -2 Situational counters

Against Aggro:

- +4 Healing cards
- +2 Defensive weapons
Remove:
- -4 Expensive weapons (too slow)
- -2 Situational cards

CHAPTER 4: MYTHOLOGY DEEP DIVE - The Stories Behind The Cards

Brahmastra - The Weapon of Ultimate Creation

The Full Story

In ancient times, warriors underwent severe penance (tapasya) to obtain divine weapons. The Brahmastra was the most coveted, for it held the power of Brahma, the creator god himself.

****The Ritual of Obtaining:****

1. Warrior performs penance for years
2. Brahma appears, pleased with devotion
3. Teaches the mantra to invoke weapon
4. Warns: "Use only against worthy opponents, or face cosmic punishment"

****Famous Uses in Epics:****

****Mahabharata - Arjuna vs Ashwatthama:****

CONTEXT: Post-war, Ashwatthama seeks revenge for his father's death

SCENE: Dark night, Ashwatthama invokes Brahmastra at Pandavas' camp

PROBLEM: Brahmastra heading toward pregnant Uttara (carrying last Pandava heir)

SOLUTION: Krishna intervenes, protects baby (future King Parikshit)

CONSEQUENCE: Ashwatthama cursed to wander earth for 3000 years with festering wounds

****Ramayana - Rama Parts the Ocean:****

CONTEXT: Need to cross ocean to reach Lanka

SCENE: Ocean god Varuna doesn't respond to prayers

ACTION: Rama threatens to dry up entire ocean with Brahmastra

RESULT: Varuna appears, begs mercy, suggests building bridge instead

LESSON: Power should be used wisely, not rashly

Why Brahmastra Works This Way in Game

****Game Mechanic:**** Power 10, Cost 5, Countered by Brahmastra or Brahmashirsha

****Mythological Basis:****

- ****High Power (10):**** Represents devastating force that can destroy armies
- ****High Cost (5):**** Reflects years of penance and spiritual preparation needed
- ****Self-Counter:**** In epics, when two Brahmastra clash, both are destroyed (mutual annihilation)
- ****Destroys Lower Weapons:**** Brahmastra is supreme among mortal weapons
- ****Rarity (Legendary):**** Only given to greatest warriors (Arjuna, Karna, Drona, Ashwatthama)

****Player Lesson:**** Like the epics teach, save your ultimate weapon for the right moment. Use it too early, face consequences. Use it wisely, achieve victory.

Naagastra - The Serpent Strike

The Serpent Kingdom in Mythology

****Background:****

Serpents (Nagas) in Hindu mythology are:

- Semi-divine beings with their own kingdom (Patala)
- Guardians of treasure and wisdom
- Both benevolent (Shesha holds universe) and vengeful (Takshaka kills kings)
- Natural enemies of Garuda (divine eagle)

Famous Naagastra Uses

****Mahabharata - Karna vs Arjuna:****

CONTEXT: Climactic battle, Karna uses all his weapons

WEAPON: Naagastra given by serpent king Takshaka

SPECIAL: Serpents of this weapon seek Arjuna specifically (revenge for Khandava forest burning)

SCENE: Thousands of fiery serpents fly toward Arjuna

COUNTER:

- Krishna (Arjuna's charioteer) pushes down chariot with his foot
- Serpents aimed at Arjuna's head hit his crown instead
- Crown destroyed, Arjuna's life saved by inches

AFTERMATH:

- Karna's mother (serpent Ashvasena) tries to help
- Karna, following warrior code, refuses help
- Shows dharmic dilemma: revenge vs righteousness

Game Mechanic Translation

****Naagastra:**** Power 6, Cost 3, Poison (2 damage/turn for 2 turns)

****Why:****

- ****Poison Effect:**** Serpent venom doesn't kill instantly but lingers
- ****Countered by Garuda:**** Natural mythological enemy relationship
- ****Medium Power:**** Not supreme but feared and respected
- ****Serpent Element:**** Own category because nagas are unique divine beings

****Strategic Lesson:**** Like serpents, be patient. You don't need overwhelming force; steady pressure wins battles.

Pashupatastra - Lord of Creatures' Weapon

The Most Sacred Weapon

****Mythology:****

Pashupatastra is Shiva's personal weapon as Pashupati (Lord of all creatures/animals). It's considered the most powerful weapon in ALL existence.

****The Story of Arjuna's Acquisition:****

MAHABHARATA - Arjuna's Penance:

SETUP:

- Pandavas exiled in forest
- Draupadi insulted, they need power for revenge
- Arjuna goes to Himalayas for divine weapons

THE TEST:

1. Arjuna performs severe penance standing on one leg, arms raised

2. Shiva appears disguised as hunter (Kirata)
3. Wild boar attacks Arjuna
4. Both Arjuna and Shiva shoot the boar simultaneously
5. They argue about who killed it first
6. Fight ensues - Arjuna vs Shiva (not knowing it's Shiva)

THE REVELATION:

- Arjuna's arrows can't harm the hunter
- Realizes this is no ordinary being
- Shiva reveals himself
- Impressed by Arjuna's skill and devotion
- Grants Pashupatastra

THE WARNING:

"This weapon can destroy entire creation. Use it only against worthy opponents.
If used against lesser enemies, it will turn and destroy YOU instead."

CONSEQUENCE:

Arjuna never uses it in entire Mahabharata war, considering no enemy worthy enough.

Game Design Philosophy

****Pashupatastra:**** Power 12, Cost 6, Counters ALL weapons except itself

****Design Reasoning:****

- ****Maximum Power:**** No weapon matches it
- ****Maximum Cost:**** Hardest to obtain, like the penance required
- ****Self-Counter Only:**** In mythology, only Shiva himself could stop his own weapon
- ****Mythic Rarity:**** Only ONE character (Arjuna) had it, never used
- ****Discard Effect:**** Opponent loses cards = weapon's overwhelming presence

****Philosophical Teaching:****

True power is restraint. Arjuna HAD the ultimate weapon but wisdom to not use it. Sometimes your greatest strength is NOT using your strength.

****Player Application:****

In game, having Pashupatastra in hand pressures opponent psychologically. They know you can destroy them anytime. Sometimes threat is stronger than execution.

The Counter System - Pancha Mahabhuta (Five Great Elements)

Ancient Elemental Philosophy

Hindu cosmology teaches the universe is made of 5 elements (Pancha Mahabhuta):

1. ****Agni (Fire)**** - Transformation and energy
2. ****Jala (Water)**** - Flow and adaptability
3. ****Vayu (Wind)**** - Movement and breath
4. ****Prithvi (Earth)**** - Stability and grounding
5. ****Akasha (Ether/Space)**** - Connection and void

Natural Cycles in Mythology

****Fire's Role:****

- ****Agni:**** God of fire, messenger between humans and gods
- ****Purification:**** Fire purifies offerings (yajna)
- ****Destruction:**** Shiva's third eye fire destroys universe at end of time
- ****Defeated By:**** Water (ocean, rain) and earth (smothering)

****Water's Role:****

- ****Varuna:**** God of cosmic order and oceans

- **Life:** All life emerges from primordial waters
- **Cleansing:** Rivers like Ganga wash away sins
- **Defeated By:** Wind (evaporation) and absorption into earth

Wind's Role:

- **Vayu:** God of wind, father of Hanuman and Bhima
- **Prana:** Life force, breath of life
- **Dispersal:** Scatters clouds, spreads seeds
- **Defeated By:** Mountains (earth blocks wind) and fire (wind feeds fire)

Earth's Role:

- **Prithvi:** Mother earth goddess
- **Stability:** Foundation for all life
- **Grounding:** Absorbs thunder, blocks wind
- **Defeated By:** Water erosion, plant roots breaking rock

Game Translation

FIRE > WIND (Fire consumes air, grows stronger)

WATER > FIRE (Water extinguishes flames)

WIND > WATER (Wind evaporates, disperses water)

EARTH > WIND (Mountains block wind)

EARTH > THUNDER (Ground absorbs lightning)

THUNDER > WATER (Electricity through water)

SPECIAL:

SERPENT (Naga) > WIND (Nagas live underground, safe from wind)

GARUDA (Divine Bird) > SERPENT (Natural predator-prey)

DIVINE > ALL ELEMENTS (Gods transcend nature)

This isn't arbitrary game design - it's 5000 years of philosophical understanding of natural cycles!

CHAPTER 5: TOURNAMENT PREPARATION

Mental Preparation

The Night Before

DO:

- ✓ Get 8 hours sleep
- ✓ Review your deck strategy once
- ✓ Prepare snacks and water
- ✓ Lay out clothes and materials
- ✓ Visualize yourself winning

DON'T:

- ✗ Completely change your deck
- ✗ Stay up late practicing
- ✗ Stress about matchups
- ✗ Consume too much caffeine
- ✗ Ignore basic needs

Tournament Day Checklist

MATERIALS: ☐ Deck in sleeves (exactly 40 cards) ☐ Sideboard (10 cards) ☐ Life Point tracker ☐ Mantra tracker

☐ Dice for randomization ☐ Pen and paper for notes ☐ Deck box ☐ Playmat (optional but recommended)

PERSONAL:

- ☐ Water bottle
- ☐ Healthy snacks (nuts, fruit)
- ☐ Phone charger
- ☐ Tournament registration confirmation
- ☐ ID if required
- ☐ Money for entry/food

MENTAL:

- ☐ Positive attitude
- ☐ Sportsmanship mindset
- ☐ Ready to learn
- ☐ Have fun!

During Tournament

Round Protocol

1. Greet Opponent:

- Shake hands

- Introduce yourself
- Wish them good game

2. Pre-Game:

- Shuffle deck thoroughly (at least 7 times)
- Offer opponent to cut/shuffle
- Roll dice for first player
- Draw opening hand (5 cards)

3. During Game:

- Announce all actions clearly
- Give opponent time to respond
- Track life/mantra accurately
- Stay focused

4. End Game:

- Shake hands win or lose
- "Good game, thank you"
- Report result to judge
- Learn from the game

Match Notes System

OPPONENT: [Name]

DECK TYPE: Fire Aggro

GAME 1: Loss

KEY CARDS: Agneyastra x2, Surya Astra, Brahmastra

MISTAKES: Didn't save Mantra for counter on turn 4

SIDEBOARD: +3 Varunastra, -2 Fire weapons

GAME 2: Win

ADJUSTMENTS: Kept water counters in hand, survived early WINNING PLAY: Varunastra chain, healed back to 15

GAME 3: Win

NOTES: They kept aggressive fire hand, I had answers

PART 2: DEVELOPER'S TECHNICAL GUIDE

CHAPTER 6: GAME DESIGN PHILOSOPHY

Core Design Pillars

Pillar 1: Mythological Authenticity

****Principle:**** Every game mechanic should reflect actual mythology

****Example:****

```python

# WRONG DESIGN:

brahmastra = Weapon(power=5, cost=2, rarity="common")

# Brahmastra was rare, powerful, high-cost in mythology!

# CORRECT DESIGN:

```
brahmastra = Weapon(
 power=10,
 cost=5,
 rarity="legendary",
 counter_mechanics={
 "countered_by": ["brahmastra", "brahmashirsha"],
 "self_destruct_on_clash": True # Mythologically accurate!
 },
 lore="Obtained through years of penance"
)
```

**\*\*Why This Matters:\*\***

- Players learn real mythology
- Creates emotional connection
- Respects cultural heritage
- Educational value

#### Pillar 2: Strategic Depth

**\*\*Principle:\*\*** Easy to learn, impossible to master

**\*\*Complexity Layers:\*\***

LAYER 1 (Beginner): Element counters (rock-paper-scissors) LAYER 2 (Intermediate): Mantra management and timing LAYER 3 (Advanced): Deck building and synergies

LAYER 4 (Expert): Meta-reading and psychology LAYER 5 (Master): Innovation and new strategies



**\*\*Design Pattern:\*\***

```
```python
```

```
class GameComplexity:
```

```
    def __init__(self):
```

```
        self.surface_level = "Play card, counter card" # 5 min to learn
```

```
        self.tactical_level = "Mantra efficiency, card advantage" # 5 hours to grasp
```

```
        self.strategic_level = "Deck building, meta adaptation" # 50 hours to master
```

```
        self.mastery_level = "Prediction, innovation" # 500+ hours
```

```
```
```

#### Pillar 3: Balanced Progression

**\*\*Principle:\*\*** Power should match cost AND mythology importance

**\*\*Balance Formula:\*\***

```
```python
```

```
def calculate_weapon_power(weapon):
```

```
    """
```

```
    Power Budget System:
```

```
    - Base power = cost * 2
```

```
    - Rarity bonus = {common: 0, rare: +1, epic: +2, legendary: +3, mythic: +4}
```

```
    - Special ability = -1 to -3 power
```

```
    - Element advantage = situational
```

```
    """
```

```
    base_power = weapon.cost * 2
```

```
    rarity_bonus = {
```

```
        "common": 0,
```

```
        "rare": 1,
```

```
        "epic": 2,
```

```
        "legendary": 3,
```

```
        "mythic": 4
```

```
    }[weapon.rarity]
```

```
    ability_cost = sum([ability.power_cost for ability in weapon.abilities])
```

```
    final_power = base_power + rarity_bonus - ability_cost
```

```
    return final_power
```

Example:

```
agneyastra = Weapon(cost=3, rarity="epic", abilities=[BurnEffect()])
```

```
# base_power = 3 * 2 = 6
```

```
# rarity_bonus = 2
```

```
# burn_effect_cost = -1
# final_power = 6 + 2 - 1 = 7 ✓ (matches our design!)
...

###
```